

A COOKBOOK FOR TYPESCRIPT DEVELOPERS

Rust for TypeScript Developers

Learn Rust fast by mapping it to the TypeScript you already know — and by meeting head-on the one big idea TS has no name for: ownership.

Rust 2021+

Cargo

Ownership

Traits & generics

Result / Option

Iterators

async / tokio

axum

Written for engineers fluent in TypeScript who want Rust — the familiar mapped fast, the genuinely new taught carefully.

Every concept: what you do in TypeScript, how Rust does it, why it's different, and the mistakes to expect.

Edition 2026 · TypeScript → Rust · syntax you'll recognize, a memory model you won't

Table of Contents

How to Use This Book	4
What's familiar, and what's genuinely new	4
The one idea: ownership instead of a garbage collector	4
Why the compiler feels like a pair programmer	5
Part 1 • Tooling — Cargo is npm + tsc + more	6
1.1 Install and the toolchain	6
1.2 Cargo, mapped to npm	6
1.3 A project's shape	7
Part 2 • Basics — Variables, Types & Control Flow	9
2.1 Variables: immutable by default	9
2.2 Types	9
2.3 Functions	10
2.4 Control flow is expressions	10
Part 3 • Ownership & Borrowing	12
3.1 The problem it solves	12
3.2 Move semantics: assignment can transfer ownership	12
3.3 Borrowing: use without owning	13
3.4 The borrowing rules (the whole borrow checker)	13
3.5 Why this is worth it	14
Part 4 • Lifetimes	15
4.1 The problem: dangling references	15
4.2 Lifetimes are mostly invisible	15
4.3 When you'll actually meet them	16
Part 5 • Structs, Enums & Pattern Matching	18
5.1 Structs	18
5.2 Enums are sum types — the star of the show	19
5.3 Pattern matching with match	19
Part 6 • Error Handling — Option & Result	21
6.1 Option: no more null	21
6.2 Result: errors are values, not throws	21
6.3 The ? operator — ergonomic propagation	22
6.4 Handling and combinators	22
Part 7 • Traits & Generics	24
7.1 Traits are interfaces	24
7.2 Deriving traits automatically	24
7.3 Generics with trait bounds	25
7.4 Static vs dynamic dispatch	25
Part 8 • Collections & Iterators	27
8.1 The collections	27
8.2 Iterators: familiar, but lazy	27

8.3 <code>iter</code> , <code>into_iter</code> , <code>iter_mut</code> — ownership shows up again	28
Part 9 · Strings — <code>String</code> vs <code>&str</code>	30
9.1 The two types	30
9.2 The practical rule	30
9.3 Building and manipulating.	31
9.4 The UTF-8 gotcha	31
Part 10 · Modules, Crates & the Ecosystem.	33
10.1 Modules	33
10.2 Crates and the ecosystem	34
Part 11 · Concurrency & <code>Async</code>	36
11.1 Threads and fearless concurrency	36
11.2 <code>Async/await</code>	37
11.3 Threads vs <code>async</code> — which when.	38
Part 12 · Building Real Things	39
12.1 Testing (built in)	39
12.2 A CLI with <code>clap</code>	39
12.3 A web service with <code>axum</code>	40
Part 13 · Cheat Sheet & Roadmap	42
13.1 TypeScript → Rust translation	42
13.2 The five mental shifts	43
13.3 The traps to expect.	43
13.4 Roadmap to fluency	43

How to Use This Book

You know TypeScript well. You think in types, generics, unions, `async/await`, and immutable-by-convention data. Rust will feel *surprisingly familiar* on the surface — expressions, closures, generics, a strong type system, a great compiler — and then, in exactly one place, profoundly new. This book leans on the familiarity to move you fast, and slows down precisely where Rust has an idea TypeScript has no word for: **ownership**.

The promise of Rust: the performance and control of C++ with the safety of a managed language, and **no garbage collector**. It catches at *compile time* the whole category of bugs — use-after-free, data races, null dereferences — that you either can't have in JS (because the GC and single thread hide them) or discover at runtime. The price is a compiler that argues with you until your program is correct. This book is about making that argument short.

Each chapter answers four questions:

1. **What do you do in TypeScript?**
2. **How does Rust do it?**
3. **Why is it different?**
4. **What mistakes should you expect?**

What's familiar, and what's genuinely new

Familiar (maps from TS quickly)	Genuinely new (no TS analog)
Static types, generics, inference	Ownership & borrowing (Part 3)
Union/enum types, pattern matching	Lifetimes (Part 4)
Interfaces → traits	No <code>null / undefined</code> → <code>Option</code> (Part 6)
<code>try/catch</code> errors → <code>Result</code>	No exceptions; errors are values
Array methods → iterators	The borrow checker enforcing all of it
<code>async/await</code>	No garbage collector; deterministic drops
npm/tsconfig → cargo	Move semantics on assignment

Roughly 70% of Rust is "TypeScript with different syntax and a stricter type system." The other 30% — the memory model — is the reason Rust exists, and it's what Parts 3 and 4 exist to teach. Don't skim those.

The one idea: ownership instead of a garbage collector

JavaScript/TypeScript uses a **garbage collector**: you create objects, share references freely, and a background process frees memory when nothing points at it. You never think

about it. Rust has no GC. Instead, every value has a single **owner**, and when the owner goes out of scope the value is freed — *deterministically*, at a known point. To let other code use a value without giving up ownership, you **borrow** it (take a reference), and the compiler's **borrow checker** enforces rules that make data races and dangling pointers impossible.

DIAGRAM

```
TypeScript:  create → share references anywhere → GC frees it "eventually"
Rust:        create → ONE owner → borrow to share → freed when owner drops
```

TYPESCRIPT MENTAL MODEL

In TS, `const b = a` makes `b` another reference to the same object; both work forever, and the GC cleans up. In Rust, for many types `let b = a` *moves* ownership — `a` is no longer usable. That single sentence is the culture shock. It's not arbitrary cruelty: it's how Rust frees memory correctly with no GC and no runtime cost. Once it clicks (Part 3), the rest of the language falls into place.

Why the compiler feels like a pair programmer

Rust's compiler error messages are famously good — they explain what's wrong and often exactly how to fix it. Early on you'll "fight the borrow checker"; treat it not as an obstacle but as a reviewer catching real bugs before they ship. The feeling of the language changes completely once you stop fighting it and start hearing it. This book's job is to get you there faster than the compiler alone would.

By the end you'll read and write idiomatic Rust: model data with structs and enums, handle errors with `Result`, abstract with traits and generics, process data with iterators, and build a real CLI and a small async web service — with the ownership model as intuition, not friction.

Part 1 · Tooling — Cargo is npm + tsc + more

Rust's tooling is one of its best features, and it will feel immediately familiar — **cargo** is your `npm` + `tsc` + test runner + formatter, all in one official tool. This short chapter gets you productive.

1.1 Install and the toolchain

`rustup` manages Rust versions (like `nvm` for Node). It installs `cargo` (the build tool/package manager) and `rustc` (the compiler you rarely call directly).

SHELL

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh  # installs rustup + cargo
rustc --version && cargo --version
rustup update  # update the toolchain
```

1.2 Cargo, mapped to npm

SHELL

```
cargo new myapp  # scaffold a project (like npm init, with a git repo)
cargo run        # compile + run
cargo build      # compile (debug); --release for optimized
cargo test       # run tests (built in – no Jest to install)
cargo add serde  # add a dependency (edits Cargo.toml)
cargo fmt        # format (like Prettier, but canonical)
cargo clippy     # lint (like ESLint, but deep and Rust-aware)
cargo check      # type-check fast without producing a binary
```

npm / TS world	Cargo / Rust
<code>package.json</code>	<code>Cargo.toml</code>
<code>package-lock.json</code>	<code>Cargo.lock</code>
<code>node_modules/</code>	<code>target/</code> (build output) + a global cache
registry: npmjs.com	crates.io
a package	a crate
<code>npm install</code>	<code>cargo build</code> (fetches deps)
<code>npm run build</code>	<code>cargo build --release</code>

npm / TS world	Cargo / Rust
<code>tsc</code> (type check)	<code>cargo check</code>
Jest / Vitest	<code>cargo test</code> (built in)
Prettier	<code>cargo fmt</code>
ESLint	<code>cargo clippy</code>
<code>npm</code>	<code>cargo install</code> + run the binary

1.3 A project's shape

DIAGRAM

```

myapp/
├─ Cargo.toml      # manifest: name, version, [dependencies]
├─ Cargo.lock      # exact resolved versions (commit it for apps)
└─ src/
   └─ main.rs      # entry point (fn main), or lib.rs for a library

```

TOML

```

# Cargo.toml
[package]
name = "myapp"
version = "0.1.0"
edition = "2021"

[dependencies]
serde = { version = "1", features = ["derive"] }
tokio = { version = "1", features = ["full"] }

```

RUST

```

// src/main.rs – the classic
fn main() {
    println!("Hello, {}!", "Rust"); // println! is a macro (note the !)
}

```

TYPESCRIPT MENTAL MODEL

You never run the compiler by hand or wire up a bundler, a test runner, a formatter, and a linter separately — cargo is all of them, blessed and consistent across every Rust project. It's the DX you wish the JS ecosystem had by default: one tool, one config file, zero setup debates. `cargo add`, `cargo test`, `cargo fmt`, `cargo clippy` will be muscle memory within a day.

PRODUCTION TIP

Run `cargo clippy` and `cargo fmt` in CI from day one — Clippy is extraordinarily good and will teach you idiomatic Rust as you go (it suggests the *Rusty* way to write things, not just style). Use `cargo check` in your editor for fast feedback; it type-checks without the slower full build. Commit `Cargo.lock` for applications (reproducible builds), omit it for libraries — same rule as `package-lock.json`.

EXERCISE 1.1

`cargo new` a project, add the `rand` crate with `cargo add rand`, and write a program that prints a random number. Run it with `cargo run`, then `cargo fmt`, `cargo clippy`, and `cargo test` (even with no tests) to feel the toolchain. Compare the experience to setting up an equivalent TypeScript project from scratch.

Part 2 · Basics — Variables, Types & Control Flow

This is the "TypeScript with different syntax" layer, and most of it will feel natural. Two things differ in spirit: variables are **immutable by default**, and Rust is **expression-oriented** (most things return a value). Get these two and the syntax is easy.

2.1 Variables: immutable by default

`let` creates an immutable binding; you opt into mutability with `mut`. This is the reverse of JS, where `let` is mutable and you opt into immutability with `const`.

TYPESCRIPT

```
const x = 5;           // immutable
let y = 10;            // mutable
y = 20;
```

RUST

```
let x = 5;              // immutable by DEFAULT
let mut y = 10;         // opt into mutability
y = 20;
// x = 6;              // compile error: cannot assign twice to immutable `x`
```

Rust also allows **shadowing** — reusing a name with a new `let`, even changing its type — which reads oddly at first but is idiomatic for transforming a value:

RUST

```
let spaces = "  ";      // &str
let spaces = spaces.len(); // now usize — same name, new value+type
```

2.2 Types

Rust is statically typed with inference, like TS — but types are never erased; they're real at runtime and drive how much memory a value takes.

RUST

```
let a: i32 = 42;         // 32-bit signed int (i8/i16/i32/i64/i128, u* for unsigned)
let b = 42;              // inferred i32
let c: f64 = 3.14;       // 64-bit float
let d: bool = true;
let e: char = ' ';       // a Unicode scalar (4 bytes), not a string
```

```
let t: (i32, &str) = (1, "hi"); // tuple
let arr: [i32; 3] = [1, 2, 3]; // fixed-size array (length is part of the type)
```

TypeScript	Rust
number	i32 / i64 / u32 / f64 /... (you pick the size + signedness)
string	String (owned) / &str (borrowed) — Part 9
boolean	bool
T[]	Vec<T> (growable) / [T; N] (fixed array)
[A, B] tuple	(A, B) tuple
null / undefined	(there is none) → Option<T> (Part 6)
any	(there is none) — the compiler won't let you off the hook

TYPESCRIPT MENTAL MODEL

Two big shifts: **there is no any** — you can't opt out of the type system, which feels strict but eliminates a whole class of runtime surprises; and **numbers have explicit sizes** — you choose i32 vs i64 vs u8 rather than one number, because Rust cares about exact memory layout. Inference means you rarely annotate locals, just like TS, but the types underneath are concrete machine types.

2.3 Functions

TYPESCRIPT

```
function add(a: number, b: number): number {
  return a + b;
}
const double = (x: number): number => x * 2;
```

RUST

```
fn add(a: i32, b: i32) -> i32 {
  a + b // no `return`, no semicolon = this is the return value
}
let double = |x: i32| x * 2; // closure (like an arrow function)
```

The last **expression** in a block is its value (no `return` needed). A line ending in `;` is a *statement* (returns nothing); omitting the `;` makes it the block's value. This trips up newcomers — an accidental `;` on the last line means "return nothing."

2.4 Control flow is expressions

`if` and `match` (Part 5) are **expressions** that return values — no ternary needed:

RUST

```
let label = if score > 90 { "A" } else { "B" }; // if returns a value

for i in 0..5 { println!("{i}"); } // 0,1,2,3,4 (a range)
for item in &items { /* iterate by reference */ }
while running { /* ... */ }
loop {
  break; } // infinite loop; break can return a
value
```

There is no C-style `for(;;)`; you iterate over ranges or collections (Part 8).

COMMON MISTAKE

The trailing-semicolon trap: `fn f() -> i32 { 5; }` fails to compile because `5;` is a statement returning `()` (unit, like `void`), not `5`. Drop the semicolon: `{ 5 }`. Also, forgetting `mut` and then trying to reassign — the compiler will tell you exactly which binding needs `mut`. Read the error; it's usually the fix verbatim.

PRODUCTION TIP

Lean into immutability — keep `let` (not `let mut`) unless you truly need to mutate. It makes code easier to reason about and lets the compiler help you, and it's the same discipline you already apply with `const` in TypeScript, just enforced by default. When you do need to change data, prefer producing a new value (iterators, Part 8) over mutating in place where it's clear.

EXERCISE 2.1

Port a few small TS functions to Rust: a `clamp(x, lo, hi)`, a function that returns the larger of two numbers using an `if` expression, and a closure stored in a variable. Deliberately add a trailing `;` to a return expression and read the compiler error. Toggle a binding between `let` and `let mut` and watch the assignment error appear and disappear.

Part 3 · Ownership & Borrowing

This is the most important chapter in the book — the one idea that makes Rust *Rust*, and the one TypeScript has no analog for. Take it slowly; everything else depends on it. Once ownership clicks, the borrow checker stops feeling like an enemy and starts feeling like a very good code reviewer.

3.1 The problem it solves

Every language must free memory somehow. JavaScript uses a **garbage collector** (a runtime that periodically finds and frees unreferenced objects — convenient, but with pauses and overhead). C uses **manual** `malloc/free` (fast, but the source of use-after-free and double-free bugs). Rust invents a third way: the **compiler** tracks who owns each value and inserts the free *for you*, at exactly the right moment, with zero runtime cost and zero chance of the classic bugs.

The mechanism is three rules:

1. Every value has exactly **one owner** (a variable).
2. When the owner goes out of scope, the value is **dropped** (freed).
3. Ownership can be **moved** or **borrowed**, but the rules above always hold.

3.2 Move semantics: assignment can transfer ownership

For types that own heap data (like `String`, `Vec`), assigning or passing by value **moves** ownership — the old variable becomes invalid. This is the single biggest surprise coming from TS.

TYPESCRIPT

```
const a = { name: "Ada" };
const b = a;           // b and a both reference the same object; both usable
console.log(a.name);   // fine
```

RUST

```
let a = String::from("Ada");
let b = a;           // ownership MOVES from a to b
// println!("{a}");   // COMPILE ERROR: borrow of moved value `a`
println!("{b}");     // fine — b owns it now
```

Why? If both `a` and `b` "owned" the string, when each went out of scope Rust would free the same memory twice (a double-free). By moving, there's always exactly one owner to do the freeing. Simple types that are cheap to copy (`i32`, `bool`, `char`, and tuples of them)

implement `Copy` and are *duplicated* instead of moved, so `let y = x;` leaves `x` usable for those.

DIAGRAM

```
let a = String::from("Ada");    a → [heap: "Ada"]
let b = a;                     a →X (invalidated)
                               b → [heap: "Ada"] ← single owner
```

3.3 Borrowing: use without owning

Moving everywhere would be painful, so you **borrow** — take a **reference** (`&`) to a value without taking ownership. The owner keeps owning; the borrow just looks.

RUST

```
fn greet(name: &String) {           // borrows: takes a reference
    println!("Hello, {name}");
}                                   // reference goes away; nothing is freed here

let a = String::from("Ada");
greet(&a);                          // lend a reference
println!("{a}");                   // a is STILL valid – greet only borrowed it
```

TYPESCRIPT

```
// TS has no distinction – everything is a shared reference by default
function greet(name: string) { console.log(`Hello, ${name}`); }
```

3.4 The borrowing rules (the whole borrow checker)

At any given time, you may have **either**:

- **any number of immutable references** (`&T`), or
- **exactly one mutable reference** (`&mut T`),

but **not both at once**. This one rule prevents data races at compile time: you can't have someone reading while someone else writes.

RUST

```
let mut s = String::from("hi");
let r1 = &s;           // ok: immutable borrow
let r2 = &s;           // ok: another immutable borrow (many readers allowed)
println!("{r1} {r2}");
let w = &mut s;        // ok NOW (r1/r2 no longer used after this point)
w.push_str(" there");
// let r3 = &s; let w2 = &mut s; // ERROR if used together: reader + writer overlap
```

TYPESCRIPT MENTAL MODEL

In TS, aliasing is invisible and unrestricted — ten parts of your code can hold the same array and mutate it, and the bug where one mutates while another iterates is a *runtime* surprise. Rust makes aliasing explicit (`&` vs `&mut`) and forbids the dangerous combination at *compile* time: many readers **or** one writer, never both. It's the "don't mutate a thing while something else is using it" rule you follow by discipline in JS — mechanically enforced.

3.5 Why this is worth it

These rules feel restrictive for a week and liberating forever. Because the compiler proves that no two threads (or even two functions) can mutate the same data simultaneously, Rust gives you **"fearless concurrency"** (Part 11) — data races are literally impossible to compile. And because ownership frees memory deterministically, there's no GC pause and no leaks from forgotten references. The bugs you spend afternoons chasing in other languages simply don't compile.

COMMON MISTAKE

"Fighting the borrow checker" by cloning everything (`a.clone()`) to dodge move errors. `.clone()` is sometimes right, but reflexively cloning to silence the compiler defeats the point (and the performance). The idiomatic fixes are usually: **borrow** (`&`) instead of taking ownership, restructure so there's a clear single owner, or return the value instead of holding a reference. When you hit a move/borrow error, ask "who should *own* this, and who just needs to *look*?" — not "how do I clone past it?"

PRODUCTION TIP

Read the compiler errors *slowly and completely* — Rust's borrow-checker messages name the exact lines where a value was moved, borrowed, and used, and frequently suggest the fix (add `&`, add `.clone()`, restructure). Early on, keep functions small and prefer **borrowing parameters** (`&T` / `&mut T`) so callers keep ownership; return owned values from functions. This "borrow in, own out" default sidesteps most beginner ownership pain.

EXERCISE 3.1

(1) Create a `String`, assign it to another variable, and try to use the first — read the "moved value" error. (2) Fix it two ways: with `.clone()`, and by borrowing with `&`. (3) Write a function that takes `&mut String` and appends to it; call it and print the mutated string afterward. (4) Try to make an immutable and a mutable reference to the same value live at once and read the borrow-checker error. You now understand the core of Rust.

Part 4 · Lifetimes

Lifetimes are the second genuinely-new concept, and the one that most intimidates newcomers — usually needlessly. A lifetime is just the compiler's way of ensuring a reference never outlives the data it points to (no dangling pointers). Most of the time you write *no* lifetime annotations at all; this chapter is about the few times you do, and why.

4.1 The problem: dangling references

A reference must never point to freed memory. In C this is a classic bug; in Rust the compiler refuses to let it happen:

RUST

```
fn dangling() -> &String {           // WON'T COMPILE
    let s = String::from("hi");
    &s                                // s is dropped at the end of the function...
}                                     // ...so the returned reference would dangle
```

The fix is usually to return the **owned** value (`String`, not `&String`) so ownership moves to the caller. The compiler catches the mistake at build time instead of letting you ship a crash.

4.2 Lifetimes are mostly invisible

Every reference has a lifetime, but the compiler infers them almost always (via "lifetime elision"). You write ordinary references and never mention lifetimes:

RUST

```
fn first_word(s: &str) -> &str {     // no annotation needed – elision handles it
    s.split(' ').next().unwrap_or("")
}
```

You only annotate when the compiler genuinely can't tell how the lifetimes of multiple references relate — most commonly when a function returns a reference derived from **more than one** reference parameter:

RUST

```
// "the returned reference lives as long as BOTH inputs" – the 'a' ties them together
fn longest<'a>(a: &'a str, b: &'a str) -> &'a str {
    if a.len() >= b.len() { a } else { b }
}
```

'a' is a *lifetime parameter* (like a generic, but for "how long a reference is valid"). It doesn't change what the code does — it's a promise you're stating to the compiler so it can verify the returned reference can't outlive its sources.

TYPESCRIPT MENTAL MODEL

TypeScript has no concept here because the garbage collector makes it unnecessary — a reference stays valid as long as anyone holds it, forever if need be. Rust frees data deterministically, so it must *prove* every reference dies before its data does. Lifetimes are that proof, written in the types. You're not managing memory by hand (the compiler does that) — you're occasionally *labeling* how references relate so the proof goes through.

4.3 When you'll actually meet them

Lifetime annotations show up mainly in three places: functions returning a reference tied to inputs (above), **structs that hold references**, and some trait implementations. A struct borrowing data must declare the lifetime:

RUST

```
struct Excerpt<'a> {
    text: &'a str,           // this struct holds a reference; it must not outlive
    `text`
}
```

For most application code — especially early on — you sidestep lifetimes entirely by **owning** data instead of holding references to it (store a `String`, not a `&str`; a `Vec<T>`, not a `&[T]`). Reach for borrowed fields only when you're optimizing to avoid copies and you understand the tradeoff.

COMMON MISTAKE

Panicking at the sight of `<'a>` and sprinkling lifetime annotations randomly to make errors go away. Ninety percent of the time the real fix is *not* a lifetime annotation — it's to **return/store an owned value** instead of a reference, which removes the lifetime question altogether. If a struct or return type is fighting you with lifetimes, first ask "should this own its data instead of borrowing it?" The answer, for application code, is usually yes.

WHEN NOT TO HOLD REFERENCES

Don't build structs full of borrowed `&str / &[T]` fields to save copies until you've measured that copying is a real cost. Borrowed fields propagate lifetime parameters through your whole API and make everything harder to use. Owning data (`String`, `Vec`) is simpler, and Rust's moves make it cheap. Optimize to references later, deliberately — start by owning.

EXERCISE 4.1

Write the `dangling` function above and read the compiler error, then fix it by returning an owned `String`. Write `longest<'a>` and call it. Then define a struct that holds a `&str` (with a lifetime) and one that holds a `String` (no lifetime), and notice how much simpler the owning version is to use. Write one sentence on when the borrowing version would be worth it.

Part 5 · Structs, Enums & Pattern Matching

Now the fun part, and where Rust's type system will delight you. Structs are TS objects/classes; **enums** are far more powerful than TS enums — they're sum types (like TS discriminated unions, but first-class), and paired with `match` they make illegal states unrepresentable. This is Rust at its most expressive.

5.1 Structs

A struct groups named fields — a TS interface/class without inheritance. Behavior goes in `impl` blocks (methods), separate from the data.

TYPESCRIPT

```
interface User { name: string; age: number; }
class UserService {
  constructor(private user: User) {}
  greet(): string { return `Hi ${this.user.name}`; }
}
```

RUST

```
struct User {
  name: String,
  age: u32,
}

impl User {
  // methods live in an impl block
  fn new(name: String, age: u32) -> Self { // "associated function" (like a
static)
    User { name, age } // field shorthand, like TS
  }
  fn greet(&self) -> String { // &self = borrows the instance
    format!("Hi {}", self.name)
  }
}

let u = User::new(String::from("Ada"), 31);
println!("{}", u.greet());
```

`&self` borrows the instance (read), `&mut self` borrows mutably (modify), `self` takes ownership (consume). There's no inheritance — you compose with structs and share behavior with **traits** (Part 7).

5.2 Enums are sum types — the star of the show

A Rust enum is a type that is *exactly one of several variants*, and each variant can **carry data**. This is TypeScript's discriminated union, built into the language and impossible to get wrong.

TYPESCRIPT

```
type Shape =
  | { kind: "circle"; radius: number }
  | { kind: "rect"; w: number; h: number };
```

RUST

```
enum Shape {
    Circle { radius: f64 },
    Rect { w: f64, h: f64 },
    // variants can hold data, or not, in tuple or struct form
}
```

The two most important types in all of Rust are enums you'll use constantly — `Option` (a value or nothing, replacing `null`) and `Result` (success or error, replacing exceptions), both covered in Part 6.

5.3 Pattern matching with `match`

`match` is `switch` on steroids: it destructures data and the compiler forces you to handle **every** case (exhaustiveness) — forget one and it won't compile.

RUST

```
fn area(shape: &Shape) -> f64 {
    match shape {
        Shape::Circle { radius } => std::f64::consts::PI * radius * radius,
        Shape::Rect { w, h } => w * h,
        // omit a variant → compile error: "non-exhaustive patterns"
    }
}
```

TYPESCRIPT

```
// TS: exhaustiveness is opt-in (a `never` trick) and easy to forget
function area(s: Shape): number {
    switch (s.kind) {
        case "circle": return Math.PI * s.radius ** 2;
        case "rect": return s.w * s.h;
        // forget a case → silent bug unless you add a `never` check
    }
}
```

`match` destructures deeply, binds variables, supports guards (`if`), ranges, and a catch-all `_`. For a single case, `if let` is the shorthand:

RUST

```
if let Shape::Circle { radius } = shape {
    println!("circle of {radius}");
}
```

TYPESCRIPT MENTAL MODEL

Rust enums are discriminated unions promoted to the core of the language, and `match` is `switch` with **compiler-enforced exhaustiveness** — the `never` exhaustiveness check you bolt onto TS switches by hand is the default here, always on. This combination ("make illegal states unrepresentable") is genuinely addictive: you model your domain as enums, and whole categories of bugs become compile errors. It's the best part of TS's type system, turned up.

PRODUCTION TIP

Model your domain with enums and let `match` exhaustiveness be your safety net: when you add a new variant (a new payment method, a new event type), the compiler walks you to *every* place that must now handle it — a fearless refactor no dynamic language can offer. Prefer struct-like enum variants (`Circle { radius }`) over positional tuples for readability, exactly as you'd prefer named fields in a TS union.

COMMON MISTAKE

Reaching for a catch-all `_ => ...` arm out of habit. It silences the exhaustiveness checker — the moment you add a new variant, the `_` swallows it silently instead of the compiler flagging the spot. Use `_` only for cases you genuinely want to treat identically and never expect to extend; otherwise list the variants so future-you gets the compile error that saves the day.

EXERCISE 5.1

Define a `Shape` enum with three variants (`circle`, `rectangle`, `triangle`), an `area` method via `match`, and a `struct` with an `impl` block. Then add a fourth variant and watch the compiler point you at the `match` that no longer covers all cases — fix it. Rewrite one `match` with a single arm as `if let`.

Part 6 · Error Handling — Option & Result

Rust has **no `null`**, **no `undefined`**, and **no exceptions**. Instead, "maybe a value" and "maybe an error" are ordinary types you handle with the type system: `Option<T>` and `Result<T, E>`. For a TypeScript developer this is the most *pleasant* difference — the billion-dollar null mistake and invisible throws are both designed out of the language.

6.1 Option: no more null

`Option<T>` is an enum with two variants: `Some(value)` or `None`. Anything that might be absent is an `Option`, and the compiler forces you to handle the `None` case before you can touch the value — so a null-dereference is impossible.

TYPESCRIPT

```
function find(id: number): User | undefined { ... }
const u = find(1);
console.log(u.name);           // runtime error if undefined (unless you checked)
```

RUST

```
fn find(id: u32) -> Option<User> { /* ... */ }
let u = find(1);
// u.name;           // WON'T COMPILE: u is Option<User>, not User
match u {
    Some(user) => println!("{}", user.name),
    None => println!("not found"),
}
```

You can't accidentally use a missing value — you must unwrap it first, which means acknowledging the `None` case. That's the whole billion-dollar bug, gone.

6.2 Result: errors are values, not throws

`Result<T, E>` is `Ok(value)` or `Err(error)`. Fallible functions return a `Result`; the caller must handle both arms. There's no invisible `throw` that propagates up an unknown number of stack frames — the type signature tells you exactly what can fail.

TYPESCRIPT

```
async function readConfig(): Promise<Config> {
    const txt = await fs.readFile("config.json", "utf8"); // may throw
    return JSON.parse(txt);                               // may throw
}
```

```
} // callers can't see what
throws
```

RUST

```
fn read_config() -> Result<Config, std::io::Error> {
    let txt = std::fs::read_to_string("config.json"?); // ? propagates the error
    let config = parse(&txt?);
    Ok(config)
}
```

6.3 The `?` operator — ergonomic propagation

The `?` after a fallible call means "if this is `Err/None`, return it from the current function; otherwise unwrap the value and continue." It gives you the *flow* of exceptions (early-return on error) with none of the invisibility — every fallible point is marked with a visible `?`.

RUST

```
fn load_user(path: &str) -> Result<User, AppError> {
    let text = std::fs::read_to_string(path)?; // ? = on error, return early
    let user: User = serde_json::from_str(&text)?;
    Ok(user) // success path is explicit
}
```

TypeScript	Rust
<code>T \ undefined / null</code>	<code>Option<T> (Some / None)</code>
<code>throw / try/catch</code>	<code>Result<T, E> (Ok / Err)</code>
error propagates invisibly	<code>?</code> marks every propagation point
<code>x?.y</code> optional chaining	<code>opt.map(...)</code> / <code>if let Some(y) = ...</code>
<code>x ?? default</code>	<code>opt.unwrap_or(default)</code>
non-null assertion <code>x!</code>	<code>.unwrap()</code> / <code>.expect("msg")</code> (can panic!)

6.4 Handling and combinators

You rarely `match` every `Option/Result` by hand — rich combinator methods read like optional chaining:

RUST

```
let name = find(1).map(|u| u.name).unwrap_or_else(|| "guest".into());
let port: u16 = std::env::var("PORT").ok()
    .and_then(|s| s.parse().ok())
    .unwrap_or(8080);
```

COMMON MISTAKE

Reaching for `.unwrap()` everywhere to get past `Option / Result` `.unwrap()` (and `.expect()`) **panic** — crash the program — if the value is `None / Err`. It's the equivalent of TS's non-null `!` assertion: fine in tests, examples, and truly-impossible cases, dangerous in production paths. In real code, propagate with `?` or handle both arms; reserve `unwrap / expect` for invariants you can prove, and prefer `.expect("why this can't fail")` so a surprise panic explains itself.

TYPESCRIPT MENTAL MODEL

`Result + ?` is what you *wish* TS errors were: the function signature declares what can fail (no hidden throws), and propagation is one visible character. `Option` is `strictNullChecks` taken to its logical end — absence is a type you must handle, so

Cannot read property 'x' of undefined cannot exist. Coming from TS, this is the change you'll miss most when you go back.

PRODUCTION TIP

For application error handling, use the `anyhow` crate (one easy error type + context for apps) and `thiserror` (to define precise library error enums) — the community standard, roughly "one big error type for apps, typed errors for libraries." Add context as errors propagate (`.context("loading config")`) so a failure reads like a trail, not a mystery. Keep `.unwrap()` out of request paths.

EXERCISE 6.1

Write `fn parse_port(s: &str) -> Option<u16>` returning `None` on bad input, using `.parse().ok()`. Write `fn load(path) -> Result<Config, io::Error>` that reads a file and uses `?`. Call both and handle the `None / Err` cases with `match` and then more concisely with `unwrap_or / map`. Finally, replace a `?` chain's error type using `anyhow::Result` and add `.context(...)`.

Part 7 · Traits & Generics

Traits are Rust's interfaces, and generics work much like TypeScript's — so this chapter is mostly familiar, with sharper edges. Traits + generics are how Rust does polymorphism and code reuse without inheritance, and they power everything from iterators to `println!`.

7.1 Traits are interfaces

A trait defines shared behavior; types `impl` it. Same idea as a TS interface a class implements — but you can implement traits for types you didn't define (including standard types), which is more powerful.

TYPESCRIPT

```
interface Summary { summarize(): string; }
class Article implements Summary {
  constructor(public title: string) {}
  summarize() { return `Article: ${this.title}`; }
}
```

RUST

```
trait Summary {
  fn summarize(&self) -> String;
  fn preview(&self) -> String { // default method (optional
    override)
    format!("{}", &self.summarize()[..10.min(self.summarize().len())])
  }
}

struct Article { title: String }
impl Summary for Article {
  fn summarize(&self) -> String { format!("Article: {}", self.title) }
}
```

Traits can provide **default methods** (like a TS interface with implementation via a base class, but cleaner). The standard library is built from traits: `Clone`, `Debug`, `Display`, `Iterator`, `PartialEq`, `Default`, and many more.

7.2 Deriving traits automatically

For common traits, `#[derive(...)]` generates the implementation — no boilerplate:

RUST

```
#[derive(Debug, Clone, PartialEq)]           // auto-implement these
struct Point { x: i32, y: i32 }

let p = Point { x: 1, y: 2 };
println!("{p:?}");                          // Debug: Point { x: 1, y: 2 }
let q = p.clone();                          // Clone
assert_eq!(p, q);                           // PartialEq
```

7.3 Generics with trait bounds

Generics parametrize over types, like TS. The difference: you constrain a type parameter with **trait bounds** (`T: Summary`) — the equivalent of TS's `<T extends Summary>` — and the compiler monomorphizes them (generates specialized code per concrete type, so there's zero runtime cost).

Typescript

```
function largest<T extends { valueOf(): number }>(items: T[]): T { ... }
```

Rust

```
fn largest<T: PartialOrd + Copy>(items: &[T]) -> T { // T must be comparable +
    copyable
    let mut max = items[0];
    for &item in items { if item > max { max = item; } }
    max
}

fn notify(item: &impl Summary) { // "some type that implements Summary"
    println!("Breaking: {}", item.summarize());
}
```

TypeScript	Rust
<code>interface X {}</code>	<code>trait X {}</code>
<code>class C implements X</code>	<code>impl X for C</code>
<code><T></code> generic	<code><T></code> generic
<code><T extends X></code>	<code><T: X></code> (trait bound)
structural typing	nominal — must explicitly <code>impl</code> the trait
<code>keyof</code> , mapped types	(no direct analog; different tools)

7.4 Static vs dynamic dispatch

Two ways to be polymorphic over a trait: **generics/ `impl Trait`** (static dispatch — resolved at compile time, fast, the default) and **`dyn Trait`** (dynamic dispatch via a trait object — like

a TS interface reference, one code path, slight runtime cost). Use `dyn` when you need a heterogeneous collection:

RUST

```
let items: Vec<Box<dyn Summary>> = vec![Box::new(article), Box::new(tweet)]; //
mixed types
for it in &items { println!("{}", it.summarize()); }
```

TYPESCRIPT MENTAL MODEL

Traits $\approx$ interfaces, generics $\approx$ generics, trait bounds $\approx$ `extends` constraints — you already think this way. Two real differences: Rust is **nominal** (a type only satisfies a trait if it explicitly `impl`s it — no accidental structural matches), and generics are **monomorphized** (zero-cost, not type-erased like TS). `#[derive]` is the delightful bit with no TS equivalent: free, correct implementations of common behavior from one line.

PRODUCTION TIP

`#[derive(Debug, Clone, PartialEq)]` on your data types by default — `Debug` makes them printable for logging/tests, and the others are cheap and handy. Prefer generics/`impl Trait` (static dispatch) for performance, and reach for `dyn Trait` only when you genuinely need a mixed collection or to break a compile-time dependency. Let Clippy suggest trait bounds; it's good at it.

WHEN NOT TO OVER-ABSTRACT WITH TRAITS

Don't invent a trait for a single implementation "for flexibility" — it's the same premature-abstraction trap as a TS interface with one implementer. Traits earn their keep when you have (or clearly will have) multiple implementers or need to write generic code over them. For one concrete type, just write the methods in an `impl` block. Add the trait when the second implementer arrives.

EXERCISE 7.1

Define a `Summary` trait with a default `preview` method, implement it for two structs, and write a generic `fn notify(item: &impl Summary)`. Add `#[derive(Debug, Clone, PartialEq)]` to a struct and use all three. Then put two different `Summary` types in a `Vec<Box<dyn Summary>>` and iterate — feeling the difference between static and dynamic dispatch.

Part 8 · Collections & Iterators

Here Rust feels like home: `Vec` is your array, `HashMap` is your `Map`, and iterator adapters (`map`, `filter`, `collect`) read almost exactly like `Array.prototype` methods. The twist — a good one — is that Rust iterators are **lazy** and **zero-cost**: they compile down to code as fast as a hand-written loop.

8.1 The collections

RUST

```
let mut v: Vec<i32> = vec![1, 2, 3]; // growable array (Array)
v.push(4);
let first = v.get(0); // Option<i32> – safe, no out-of-bounds
// crash
let x = v[0]; // direct index (panics if out of range)

use std::collections::HashMap;
let mut scores: HashMap<String, i32> = HashMap::new(); // Map
scores.insert("ada".into(), 10);
scores.entry("ada".into()).and_modify(|s| *s += 1).or_insert(0); // upsert
if let Some(s) = scores.get("ada") { println!("{}", s); }
```

TypeScript	Rust
<code>T[]</code> / <code>Array<T></code>	<code>Vec<T></code>
<code>Map<K,V></code>	<code>HashMap<K, V></code>
<code>Set<T></code>	<code>HashSet<T></code>
<code>arr[i]</code> (maybe undefined)	<code>v.get(i) → Option<&T></code> (safe) / <code>v[i]</code> (panics)
<code>Object</code> as dict	<code>HashMap</code> (or a struct if keys are known)

8.2 Iterators: familiar, but lazy

The adapter methods you know map over directly — but they don't run until you **consume** them (with `collect`, `sum`, `for`, etc.). This laziness means chains fuse into one efficient pass with no intermediate arrays allocated.

TYPESCRIPT

```
const result = users
  .filter(u => u.active)
  .map(u => u.name)
  .slice(0, 10);
```

RUST

```
let result: Vec<String> = users.iter()           // start an iterator (borrows)
    .filter(|u| u.active)                       // lazy
    .map(|u| u.name.clone())                    // lazy
    .take(10)                                   // lazy
    .collect();                                // consume → build the Vec
```

TS array method	Rust iterator adapter
<code>map</code>	<code>map</code>
<code>filter</code>	<code>filter</code>
<code>reduce</code>	<code>fold</code>
<code>forEach</code>	<code>for_each</code> (or a <code>for</code> loop)
<code>find</code>	<code>find</code>
<code>some / every</code>	<code>any / all</code>
<code>slice(0,n)</code>	<code>take(n)</code>
<code>flatMap</code>	<code>flat_map</code>
<code>sort</code>	<code>sort()</code> (on a <code>Vec</code> , in place)
— (build result)	<code>collect()</code> (the consuming step)

8.3 `iter`, `into_iter`, `iter_mut` — ownership shows up again

Because of ownership (Part 3), there are three ways to iterate, differing in what you get:

RUST

```
for x in &v { } // v.iter() → &T (borrow; v still usable after)
for x in &mut v { } // v.iter_mut() → &mut T (mutate in place)
for x in v { } // v.into_iter() → T (consumes v; it's moved)
```

`.iter()` borrows (most common), `.iter_mut()` lets you modify each element, and `.into_iter()` takes ownership (the `Vec` is consumed). This is just ownership applied to iteration — pick based on whether you need to keep the collection.

TYPESCRIPT MENTAL MODEL

Iterator chains read like `Array.prototype` chains, so you'll be productive immediately — but two upgrades: they're **lazy** (nothing runs until `collect / sum / for`, so there are no throwaway intermediate arrays), and they're **zero-cost** (the compiler fuses the chain into a tight loop as fast as writing it by hand). You get functional-style expressiveness with hand-loop performance — the thing that's a real tradeoff in JS is free here.

COMMON MISTAKE

Forgetting to **consume** the iterator — `v.iter().map(f);` does *nothing* (and Rust warns you), because `map` is lazy and you never collected or looped. Add a consumer:

`.collect()`, `.sum()`, `.for_each(...)`, or a `for`. The other classic: indexing with `v[i]` on possibly-empty data and panicking — prefer `v.get(i)` which returns `Option` and forces you to handle the empty case.

PRODUCTION TIP

Prefer iterator chains over manual index loops — they're idiomatic, harder to get wrong (no off-by-one, no bounds panics), and just as fast. Use `.collect::<Vec<_>>()` (turbofish) or a typed binding to tell `collect` what to build. And when you don't need the collection afterward, `into_iter()` to consume it and avoid clones — let ownership work for you.

EXERCISE 8.1

Port a `filter().map().slice()` chain from TS to a Rust iterator chain ending in `collect()`. Build a `HashMap` word-count from a sentence using `entry().or_insert()`. Then iterate the same `Vec` three ways (`&v`, `&mut v` to double each element, and `v` to consume it) and note which leave `v` usable afterward.

Part 9 · Strings — String vs &str

Strings are the one everyday topic where Rust surprises TypeScript developers, because Rust has **two** string types where TS has one, and the distinction is ownership again. Five minutes here saves you hours of confusion.

9.1 The two types

- **String** — an **owned**, growable, heap-allocated string. You own it, you can modify it, it's freed when it drops. Think "the owner of some text."
- **&str** (a "string slice") — a **borrowed** view into string data (owned by a **String**, or a string literal baked into the binary). It doesn't own anything; it's a window.

RUST

```
let owned: String = String::from("hello"); // owns heap data
let literal: &str = "hello";              // borrowed view (a &'static str)
let slice: &str = &owned[0..3];            // "hel" — a borrowed view into owned
let back: String = literal.to_string();    // &str → String (allocates)
```

TYPESCRIPT

```
// TypeScript has one string type; no owned/borrowed distinction
const s: string = "hello";
```

9.2 The practical rule

You'll internalize this fast:

- Take **&str** as a function parameter — it accepts both **String** (auto-borrows) and string literals, so it's the most flexible input type.
- Return / store **String** when you own the data (built it, need to keep it).

RUST

```
fn greet(name: &str) -> String {           // &str in (flexible), String out (owned)
    format!("Hello, {name}!")
}
greet("Ada");                             // literal works
greet(&String::from("Ada"));               // String auto-borrows to &str
```

You have / want	Use
A function parameter (read text)	&str
A returned/stored, owned string	String

You have / want	Use
A string literal	<code>&str</code> (it's <code>&'static str</code>)
Build up text piecewise	<code>String</code> + <code>push_str</code> / <code>format!</code>
A view into part of a string	<code>&str</code> slice

9.3 Building and manipulating

RUST

```
let mut s = String::new();
s.push_str("Hello");           // append a &str
s.push(' ');                   // append a char
s += "world";                  // also append
let joined = format!("{s}!"); // format! builds a new String (like a template literal)
let n = s.len();               // length in BYTES, not chars
for c in s.chars() { }        // iterate Unicode scalar values
```

9.4 The UTF-8 gotcha

Rust strings are **UTF-8**, and Rust makes you respect that. `s.len()` is the byte length, not the character count, and you **cannot** index a string by character position (`s[0]` doesn't compile) because a "character" can be multiple bytes. Iterate with `.chars()` (Unicode scalars) or `.bytes()`, and slice only on valid byte boundaries.

RUST

```
let s = "héllö";
// s[0];           // ERROR: strings can't be indexed by integer
s.chars().nth(0);  // Some('h') – but this walks the string, 0(n)
s.len();           // 6 (é is 2 bytes), not 5
```

TYPESCRIPT MENTAL MODEL

JS strings hide UTF-16 and let you index by code unit (with its own emoji surprises); Rust refuses to pretend text is a simple array of characters, because it isn't. The `String` vs `&str` split is just ownership from Part 3 applied to text: `String` is the owner (`Vec<u8>` under the hood), `&str` is a borrowed slice of it. "`&str` for parameters, `String` for ownership" is the rule that makes it click.

COMMON MISTAKE

Taking `String` as a function parameter when you only read from it — this forces callers to give up ownership (or clone). Take `&str` instead; it accepts everything. The other classic: trying to index a string (`s[0]`) or assuming `.len()` is character count. Use `.chars()` for characters, and slice on byte boundaries you know are valid.

PRODUCTION TIP

Default your function signatures to `&str` inputs and `String` (or `-> impl Display`) outputs — it's the most ergonomic, clone-free API. Use `format!` for building strings (it's the template-literal equivalent), and reach for `&str` slices to pass around substrings without copying. When you truly need character-aware logic, `.chars()` is your tool, mindful that it's $O(n)$ to index.

EXERCISE 9.1

Write `fn shout(s: &str) -> String` returning the uppercased string with "!" appended, and call it with both a literal and a `String`. Print the byte length vs character count of a string containing an accented letter or emoji, and iterate it with `.chars()`. Try to index it by integer and read the compiler error.

Part 10 · Modules, Crates & the Ecosystem

Organizing code and pulling in dependencies maps cleanly from the npm world, with different keywords. This short chapter covers modules (`mod` / `use`), crates (packages), and the handful of libraries you'll reach for constantly.

10.1 Modules

A **module** groups related code and controls visibility. Everything is **private by default**; you expose items with `pub`. Modules can be inline, in separate files, or in directories.

RUST

```
// src/main.rs
mod billing;                // loads src/billing.rs (or src/billing/mod.rs)

use billing::charge;        // bring a name into scope (like an import)

fn main() { charge(100); }
```

RUST

```
// src/billing.rs
pub fn charge(cents: u32) {    // pub = visible outside this module
    let fee = internal_fee(cents); // private helper, same module
    // ...
}
fn internal_fee(c: u32) -> u32 { c / 50 } // private
```

TypeScript	Rust
<code>export</code>	<code>pub</code>
<code>import { x } from "./m"</code>	<code>use crate::m::x;</code>
a file is a module	<code>mod m;</code> declares one (file or inline)
<code>import * as m</code>	<code>use crate::m; then m::x</code>
default private? (no)	yes — private by default , opt in with <code>pub</code>
barrel <code>index.ts</code>	<code>mod.rs</code> / re-exports with <code>pub use</code>

Paths: `crate::` (your crate root), `self::` (current module), `super::` (parent), and a crate name for dependencies (`serde::...`).

10.2 Crates and the ecosystem

A **crate** is a package (a library or binary). You add dependencies from **crates.io** with `cargo add`, and the compiler + `Cargo.lock` handle resolution. The ecosystem is smaller than npm but higher-signal — a handful of crates are near-universal:

Need	Crate	TS-world analog
Serialization (JSON, etc.)	serde / <code>serde_json</code>	zod-ish + <code>JSON.parse</code> (typed)
Async runtime	tokio	the Node event loop, as a library
HTTP server	axum / <code>actix-web</code>	Express / Fastify
HTTP client	reqwest	axios / fetch
Error handling	anyhow / <code>thiserror</code>	(Part 6)
CLI parsing	clap	commander / yargs
Logging / tracing	tracing	pino / winston
Time	chrono / <code>time</code>	date-fns / luxon
Testing extras	built-in + proptest	Jest + fast-check

RUST

```
// serde: derive serialization on your types (like typed JSON, checked at compile
time)
use serde::{Serialize, Deserialize};

#[derive(Serialize, Deserialize, Debug)]
struct User { name: String, age: u32 }

let json = serde_json::to_string(&user)?;           // struct → JSON
let user: User = serde_json::from_str(&json)?;     // JSON → struct
(validated)
```

TYPESCRIPT MENTAL MODEL

`mod / use / pub` are `import / export` with the defaults flipped to private-first, which nudges you toward small public APIs. `serde` deserves special mention: it's like `zod + JSON.parse` but the schema is your *actual struct*, checked at compile time and generated with one `#[derive]` — no separate schema to keep in sync. Coming from TS, `serde` is a highlight.

PRODUCTION TIP

Lean on the "blessed" crates rather than hand-rolling — `serde` for anything data-shaped, `tokio + axum` for services, `anyhow / thiserror` for errors, `clap` for CLIs, `tracing` for logs. They're mature, well-documented, and interoperate. Check a crate's downloads/recent activity on `crates.io` (the ecosystem is smaller, so quality varies), and pin versions in `Cargo.toml`.

EXERCISE 10.1

Split a program into a `mod` in a separate file with one `pub` function and one private helper; call it via `use`. Add `serde + serde_json`, derive `Serialize / Deserialize` on a struct, and round-trip it to JSON and back. Notice you wrote no schema — the struct *is* the schema.

Part 11 · Concurrency & Async

Rust's tagline for concurrency is "**fearless**" — the ownership rules from Part 3 make data races a *compile error*, so you can use real threads without the class of bugs that makes concurrency terrifying elsewhere. And its `async/await` will look instantly familiar, with one twist: you bring your own runtime.

11.1 Threads and fearless concurrency

Unlike Node's single thread, Rust has real OS threads and can use all your cores. The borrow checker extends to threads via two marker traits — `Send` (safe to move to another thread) and `Sync` (safe to share between threads) — which the compiler checks automatically. You literally cannot compile a program that shares mutable data across threads without synchronization.

RUST

```
use std::thread;

let handle = thread::spawn(|| {           // spawn a real OS thread
    heavy_computation()
});
let other = do_other_work();
let result = handle.join().unwrap();      // wait for the thread, get its result
```

To share mutable state across threads, the types force correctness: `Arc<T>` (atomic reference count — shared ownership across threads) wrapping a `Mutex<T>` (mutual exclusion). Forget the lock and it won't compile.

RUST

```
use std::sync::{Arc, Mutex};
let counter = Arc::new(Mutex::new(0));
let c = Arc::clone(&counter);
thread::spawn(move || { *c.lock().unwrap() += 1; }); // must lock to touch it
```

TYPESCRIPT MENTAL MODEL

Node hands you concurrency by *avoiding* shared-memory threads entirely — one event loop, and CPU work goes to worker threads that communicate by copying messages. Rust gives you actual shared-memory threading but makes the dangerous part (two threads touching the same data) a **compile error** via ownership + `Send / Sync`. The data-race bugs you never have to think about in single-threaded JS, Rust also lets you not think about — because they won't build.

11.2 Async/await

For high-concurrency I/O (servers, many network calls), Rust has `async/await` that reads just like TypeScript's. The difference: JS ships an event loop in the runtime; Rust keeps the language runtime-free, so you add an **async runtime** crate — almost always **tokio**.

Typescript

```
async function fetchUser(id: number): Promise<User> {
  const res = await fetch(`/users/${id}`);
  return res.json();
}
```

Rust

```
async fn fetch_user(id: u32) -> Result<User, request::Error> {
  let res = request::get(format!("https://api/users/{id}")).await?;
  res.json().await
}

#[tokio::main]                                // the runtime that drives async (like Node's
loop)
async fn main() {
  let user = fetch_user(1).await.unwrap();
}
```

Concurrent awaits map directly:

Typescript

```
const [a, b] = await Promise.all([fetchUser(1), fetchUser(2)]);
```

Rust

```
let (a, b) = tokio::join!(fetch_user(1), fetch_user(2)); // run concurrently
```

Node / TS	Rust
<code>async function</code> / <code>await</code>	<code>async fn</code> / <code>.await</code> (postfix!)
a <code>Promise</code>	a <code>Future</code> (lazy — does nothing until awaited)
<code>Promise.all</code>	<code>tokio::join!</code> / <code>futures::join_all</code>
built-in event loop	a runtime crate: tokio
worker threads (CPU)	<code>thread::spawn</code> / <code>tokio::task::spawn_blocking</code>
<code>EventEmitter</code>	<code>channels</code> (<code>tokio::sync::mpsc</code>)

Note `.await` is **postfix** (`future.await`), which chains nicely, and a `Future` is **lazy** — it doesn't start running until awaited or spawned (unlike a JS `Promise`, which starts immediately).

11.3 Threads vs async — which when

Same rule as everywhere: **async for many concurrent I/O waits** (a web server handling thousands of connections), **threads for CPU-bound parallelism** (using all cores for computation). Don't make CPU-heavy work `async` (it blocks the runtime, just like blocking the Node loop); don't spawn thousands of OS threads for I/O (that's what async is for).

COMMON MISTAKE

Two async traps for JS developers: (1) expecting a `Future` to run when created — it doesn't; you must `.await` it or `tokio::spawn` it, or it never executes (Rust warns: "unused `Future` that must be used"). (2) doing blocking or CPU-heavy work inside an `async fn`, which stalls the whole tokio runtime — offload it with `spawn_blocking`, exactly as you'd offload CPU work off the Node event loop.

PRODUCTION TIP

For services, `tokio + axum + request` is the standard, ergonomic stack. Reach for threads (`std::thread`, or `rayon` for data-parallelism) when you're CPU-bound and want all cores; reach for async when you're I/O-bound and want high concurrency on few threads. And trust the compiler: if it won't let you share data across threads, it's preventing a real race — wrap it in `Arc<Mutex<...>>` (or use a channel) rather than fighting it.

EXERCISE 11.1

(1) Spawn two threads that each compute something and `join` their results. (2) Share a counter across threads with `Arc<Mutex<i32>>` and increment it from several threads. (3) Write two `async fn`s and run them concurrently with `tokio::join!` under `#[tokio::main]`. (4) Create a `Future` without awaiting it and read the compiler's warning.

Part 12 · Building Real Things

Concepts stick when you build. This part assembles what you've learned into two real programs — a CLI and a small web service — plus testing, so you finish with templates you can actually start projects from.

12.1 Testing (built in)

`cargo test` runs tests with no framework to install. Tests live next to the code (unit) or in a `tests/` directory (integration).

RUST

```
fn add(a: i32, b: i32) -> i32 { a + b }

#[cfg(test)]                                // compiled only for tests
mod tests {
    use super::*;
    #[test]
    fn adds() {
        assert_eq!(add(2, 3), 5);
        assert!(add(2, 3) > 0);
    }
    #[test]
    fn handles_error() {
        assert!("x".parse::<i32>().is_err());
    }
}
```

`assert_eq!`, `assert!`, and `#[should_panic]` cover most needs; tests that return `Result` let you use `?`. Run a subset with `cargo test adds`.

12.2 A CLI with clap

`clap` parses arguments from a struct via `derive` — typed, validated, with `--help` generated for free.

RUST

```
use clap::Parser;

#[derive(Parser)]
#[command(version, about = "Greet someone")]
struct Args {
    name: String,                // positional arg
    #[arg(short, long, default_value_t = 1)]
```

```

    times: u32,                                // --times / -t, defaults to 1
}

fn main() {
    let args = Args::parse();                    // parses argv; errors + --help handled
    for _ in 0..args.times {
        println!("Hello, {}!", args.name);
    }
}

```

SHELL

```
cargo run -- Ada --times 3      # Hello, Ada! x3    (args after -- go to your program)
```

12.3 A web service with axum

`axum` (on `tokio`) is the Express of Rust: routes to handler functions, typed extractors for the body/params, and `serde` for JSON.

RUST

```

use axum::{routing::{get, post}, Router, Json};
use serde::{Serialize, Deserialize};

#[derive(Deserialize)]
struct CreateUser { name: String }
#[derive(Serialize)]
struct User { id: u64, name: String }

async fn health() -> &'static str { "ok" }

async fn create_user(Json(body): Json<CreateUser>) -> Json<User> { // typed JSON in
    Json(User { id: 1, name: body.name })                          // typed JSON out
}

#[tokio::main]
async fn main() {
    let app = Router::new()
        .route("/health", get(health))
        .route("/users", post(create_user));
    let listener = tokio::net::TcpListener::bind("0.0.0.0:3000").await.unwrap();
    axum::serve(listener, app).await.unwrap();
}

```

TYPESCRIPT

```

// the Express shape you already know, for comparison
app.get("/health", (_req, res) => res.send("ok"));
app.post("/users", (req, res) => res.json({ id: 1, name: req.body.name }));

```

The request body is deserialized into your struct by `serde` (invalid JSON → a `400` automatically), and your response struct is serialized back — end-to-end typed, validated at the boundary, with no manual parsing.

TYPESCRIPT MENTAL MODEL

axum is Express/Fastify with the ergonomics you'd want: handlers are async functions, `Json<T>` extractors give you a typed, validated body (no `req.body: any`), and responses serialize from typed structs. `clap` is commander/yargs but the arg spec is a struct with `--help` generated for free. You're writing the same kinds of programs you write in Node — the shapes transfer directly.

PRODUCTION TIP

Start services from the `tokio + axum + serde` template above and add `tracing` for structured logs, `anyhow/thiserror` for errors (Part 6), and `sqlx` if you need a database (compile-time-checked SQL — a genuine treat). Write tests as you go (`cargo test` is free and fast), and run `cargo clippy / cargo fmt` in CI. A Rust service compiles to a single small static binary — trivial to containerize (a `FROM scratch` or distroless image, per the Docker cookbook).

EXERCISE 12.1

Build both: (1) a `clap` CLI that takes a name and a count and greets accordingly, with a unit test for its core function; (2) an `axum` service with a `GET /health` and a `POST /echo` that returns the posted JSON typed through a struct. Run the service, hit it with `curl`, and add one integration test in `tests/`. You've now built the two things most backends are made of, in Rust.

Part 13 · Cheat Sheet & Roadmap

The whole book condensed: the TypeScript→Rust translation table, the mental shifts, the traps, and a plan to reach fluency.

13.1 TypeScript → Rust translation

TypeScript	Rust
<code>const x = 1</code>	<code>let x = 1;</code> (immutable by default)
<code>let x = 1</code> (mutable)	<code>let mut x = 1;</code>
<code>number</code>	<code>i32</code> / <code>i64</code> / <code>u32</code> / <code>f64</code> /...
<code>string</code>	<code>String</code> (owned) / <code>&str</code> (borrowed)
<code>T[]</code> / <code>Array<T></code>	<code>Vec<T></code>
<code>Map</code> / <code>Set</code>	<code>HashMap</code> / <code>HashSet</code>
<code>[A, B]</code> tuple	<code>(A, B)</code>
<code>interface</code>	<code>trait</code>
<code>class implements X</code>	<code>impl X for C</code>
<code>class</code> (data)	<code>struct + impl</code>
discriminated union	<code>enum</code> (with data)
<code>switch</code> (+ <code>never</code>)	<code>match</code> (exhaustive by default)
<code>T</code> \ <code>undefined</code> / <code>null</code>	<code>Option<T></code>
<code>throw</code> / <code>try/catch</code>	<code>Result<T, E> + ?</code>
<code>x?.y</code> / <code>x ?? d</code>	<code>opt.map(...)</code> / <code>opt.unwrap_or(d)</code>
<code>x!</code> (non-null)	<code>.unwrap()</code> / <code>.expect()</code> (can panic)
<code><T extends X></code>	<code><T: X></code>
<code>async function</code> / <code>await</code>	<code>async fn</code> / <code>.await</code>
<code>Promise<T></code>	<code>Future<Output = T></code> (+ tokio)
<code>Promise.all</code>	<code>tokio::join!</code>
<code>arr.map/filter/reduce</code>	<code>.iter().map/filter/fold</code> (+ <code>.collect()</code>)
<code>import</code> / <code>export</code>	<code>use</code> / <code>pub</code>
<code>npm</code> / <code>package.json</code>	<code>cargo</code> / <code>Cargo.toml</code>
Jest, Prettier, ESLint	<code>cargo test</code> / <code>fmt</code> / <code>clippy</code>
Express / axios / zod	<code>axum</code> / <code>reqwest</code> / <code>serde</code>

13.2 The five mental shifts

1. **Ownership, not GC.** One owner frees the value; assignment often *moves*. Borrow (`&`) to share. (Part 3)
2. **Errors are values.** No exceptions, no `null` — `Result` and `Option`, with `?` to propagate. (Part 6)
3. **Exhaustive by default.** `match` + enums force you to handle every case; the compiler is a reviewer, not an obstacle. (Part 5)
4. **Immutable by default.** `let` is `const`; opt into change with `mut`. (Part 2)
5. **Two string types.** `&str` to borrow/pass, `String` to own. (Part 9)

13.3 The traps to expect

Trap	Fix
"value moved" after assignment	borrow (<code>&</code>), or restructure ownership (P3)
Cloning everything to dodge the borrow checker	ask "who owns vs who looks" (P3)
Trailing <code>;</code> making a function return <code>()</code>	drop the semicolon on the last expression (P2)
Panicking on <code>.unwrap()</code> in prod	propagate with <code>?</code> / handle both arms (P6)
Catch-all <code>_</code> hiding new enum variants	list variants for exhaustiveness (P5)
Iterator that "does nothing"	consume it (<code>collect</code> / <code>for</code> / <code>sum</code>) (P8)
Taking <code>String</code> param when you only read	take <code>&str</code> (P9)
Fearing <code><'a></code> lifetimes	usually: own the data instead (P4)
<code>Future</code> that never runs	<code>.await</code> or <code>spawn</code> it (P11)

13.4 Roadmap to fluency

Week 1 — Familiar ground (Parts 1-2, 5, 7). Tooling, basics, structs/enums, traits. Port 20 small TS functions and a couple of types. *You can:* write and run typed Rust that looks like the TS you know.

Week 2 — The new model (Parts 3-4, 9). Ownership, borrowing, lifetimes, strings. Do every exercise; deliberately trigger and fix borrow errors. *You can:* read borrow-checker errors and fix them by adjusting ownership, not cloning.

Week 3 — Idiomatic Rust (Parts 6, 8, 10). `Result` / `Option` / `?`, iterators, modules/crates + `serde`. Rewrite a data-processing script the Rusty way. *You can:* handle errors and transform data idiomatically, use the ecosystem.

Week 4 — Real programs (Parts 11-12). Concurrency/async, then build a `clap` CLI and an `axum` service with tests. *You can:* ship a small Rust binary or service end to end.

Then keep going: read the compiler's suggestions and Clippy's lints (they teach idiom), work through *the* free book ("The Rust Programming Language", a.k.a. "the Book") and *Rustlings* exercises, and build something real — a CLI tool you'd use, or a service to replace

a hot Node endpoint. The borrow checker that fought you in week 2 becomes the pair-programmer that lets you refactor fearlessly by week 4.

YOU'RE READY

Rust rewards the TypeScript developer twice: most of it is the strong-typed, generic, pattern-matching, async programming you already enjoy — and the part that's new (ownership) buys you memory safety *and* fearless concurrency with no garbage collector and no runtime cost. Lean on the familiarity to move fast, respect the ownership chapter enough to truly learn it, and let the compiler teach you the rest. Welcome to Rust.